

Recherche et sauvetage autonome avec un seul TurtleBot 4 : une mission en deux phases orchestrée par un arbre de comportement

Équipe RobotZ

Julien GIMENEZ Hugo FANCHINI Paul CINTRA Yimou ZHANG

{julien.gimenez, hugo.fanchini, paul.cintra, yimou.zhang}@telecom-paris.fr

Projet B - IA712 Robotique mobile, projet final - Télécom Paris

Dépôt : <https://github.com/YiZeems/autonomous-search-and-rescue>

Résumé—Un unique TurtleBot 4 explore de manière autonome une arène de catastrophe inconnue et cloisonnée, construit une carte par SLAM et localise quatre « victimes » (AprilTags) en projetant les détections de la caméra dans le repère global `map` avec `tf2`. La décision est confiée à un *arbre de comportement* (et non à une machine à états finis, comme l'exige le sujet [1]) qui orchestre une **mission en deux phases** : d'abord l'exploration par frontières, puis une inspection des pièces dérivée de la carte. Nous montrons que l'exploration pure achève la carte mais manque structurellement les victimes, car la caméra n'approche jamais assez près des tags muraux ; une seconde phase autonome, qui dérive une pose d'inspection par pièce découverte, comble cet écart. L'exécution finale autonome en un clic atteint **97,35 % de couverture et 4/4 victimes**. Cet article présente l'architecture, la théorie sous-jacente et une évaluation quantitative.

1 Introduction

La robotique de recherche et sauvetage vise à envoyer des machines autonomes dans des zones de catastrophe où l'humain ne peut pénétrer sans risque. Le scénario « Projet B » d'IA712 [1] formalise cela comme un problème concret : un robot mobile est déposé dans un bâtiment simulé et cloisonné qu'il ne connaît pas *a priori*, et doit *explorer de façon autonome*, construire une carte cohérente, *localiser les victimes* représentées par des marqueurs visuels, et *reporter leurs coordonnées* dans un repère global fixe - le tout sans intervention humaine. Le sujet impose plusieurs contraintes strictes : l'exploration doit être autonome (frontières ou RRT), la carte doit couvrir plus de 90 % de la surface et être annotée de chaque victime, les positions doivent être projetées dans le repère `map`, la chaîne doit se lancer en *un clic*, et surtout la décision doit être un arbre de comportement plutôt qu'une machine à états finis.

Notre plateforme est un TurtleBot 4 (base Create 3, RPLIDAR A1M8, caméra OAK-D) simulé sous Ignition Gazebo Fortress et ROS 2 Humble. L'arène `rescue_arena` est un bâtiment de quatre pièces portant un AprilTag « victime » sur un mur extérieur de chacune. À partir d'un unique lancement, le robot cartographie le bâtiment par SLAM, visite chaque pièce, détecte les quatre tags, les projette dans le repère `map` et s'arrête sur une carte annotée propre. L'idée centrale de ce travail est que la complétude de la carte et la couverture des victimes sont des objectifs *distincts* : un robot peut cartographier parfaitement une pièce depuis sa porte sans jamais approcher sa caméra du tag mural. Nous découpons donc la mission en deux phases autonomes coordonnées par un arbre de comportement, ce que nous évaluons ci-dessous.

2 Système et méthode

Architecture. L'espace de travail est divisé en quatre paquets ROS 2 pour permettre le travail en parallèle : `rescue_bringup` (lancement en un clic et configurations Nav2/SLAM/AprilTag), `rescue_robot` (un « mégapaquet » Python regroupant exploration, perception, inspection et résultats), `rescue_world` (l'arène Ignition et

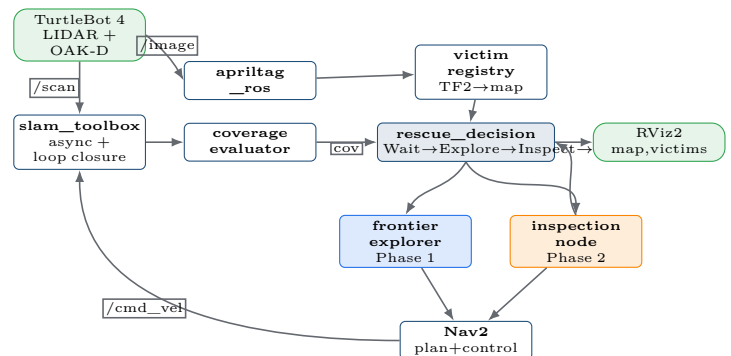


Figure 1 – Architecture du système et flux de topics. L'arbre de comportement (`rescue_decision`) orchestre l'explorateur de Phase 1 et l'inspecteur de Phase 2 ; le SLAM alimente la carte de tous.

les cibles AprilTag en voxels, source unique de vérité pour le placement des victimes), et `rescue_decision` (l'exécuteur C++ `BehaviorTree.CPP` v3). La Fig. 1 montre le flux de topics. L'arbre de comportement est l'*orchestrateur* : il active l'explorateur de Phase 1 et l'inspecteur de Phase 2 via les topics `/mission/*`, tandis que les nœuds Python font le travail. Le SLAM alimente en continu la carte vers l'exploration, la couverture, Nav2 et l'inspection.

Chaîne de perception (TF2). Les victimes sont des AprilTags `tag36h11` ; le sujet exclut explicitement une détection humaine de type YOLO. Chaque détection fournit une transformation `caméra` → `victimi` ; `victim_registry` la chaîne via `tf2` en `caméra` → `victimi` → `map`, déduplique par identifiant de tag et sauvegarde `results/victims.json`. Cela réalise l'exigence « reporter les coordonnées dans le repère global » issue du cours sur les repères.

Exploration et planification. Une *frontière* est la limite entre cellules libres et inconnues [2]. La sélection gloutonne prend la frontière la plus proche ; la sélection par *gain d'information* maximise $g(f) = \text{gain}(f) - \lambda \cdot \text{coût}(f)$, où le coût est la *longueur du chemin* Nav2. Une liste noire indexée par coordonnée monde quantifiée écarte définitivement une frontière morte dès que Nav2 échoue à l'atteindre ou qu'elle est re-choisie **3 fois sans gain de couverture**. Pour la planification globale nous avons conservé Dijkstra/NavFn plutôt que A*/Smac : dans une arène cloison-

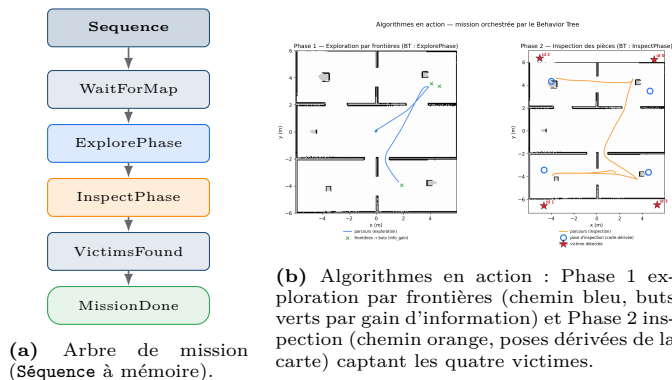


Figure 2 – La mission en deux phases orchestrée par l'arbre de comportement. Les murs sont dessinés par-dessus le chemin, qui traverse donc visiblement les portes.

née le robot démarre souvent dans la couche d'inflation, où A^* échoue (« départ en zone létale ») alors que NavFn le tolère ; le contrôleur est DWB.

La mission en deux phases. L'exploration pure achève la carte mais plafonne la détection, car la caméra (portée $\sim 2m$) n'atteint jamais un tag mural que le LIDAR cartographie depuis la porte (Fig. 2). La solution est une seconde phase *autonome* : à partir de la carte qu'il vient de bâtir, *inspection_node* dérive une pose par pièce découverte - face au mur le plus proche, ramenée à une cellule navigable et *sans* coordonnées de victime - puis s'y rend et balaye la caméra. La mission est la **Séquence** à mémoire `WaitForMap` → `ExplorePhase` → `InspectPhase` → `VictimsFound` → `PublishMissionDone`. Étant une séquence à mémoire, l'arbre reprend au nœud `RUNNING` et n'avance qu'au `SUCCESS`, donnant une vraie progression réactive de phases : c'est l'esprit de l'exigence « décision = arbre de comportement, pas MEF ».

3 Résultats

L'exécution autonome finale (archivée sous `results/examples/118_nominal`) atteint **97,35 % de couverture** du bâtiment - bien au-delà de la cible de 90 % - et détecte **4/4 victimes** en une seule mission continue et entièrement autonome, lancée par une commande, `ros2 launch rescue_bringup bringup_tb4.launch.py`. La Fig. 3 présente la courbe de couverture, la trajectoire réelle dans le repère `map` et la carte finale annotée. La trajectoire est le chemin réellement enregistré : les murs sont tracés par-dessus et aucun des 568 segments ne traverse un mur, le robot enfilant donc visiblement les portes. La Fig. 4 montre la chronologie de la mission avec les deux phases de l'arbre en bandes colorées, confirmant que les quatre détections surviennent durant le balayage de la Phase 2.

Banc d'essai quantitatif d'exploration. Un banc d'essai bonus ($n=3$) confirme la théorie : la sélection par gain d'information atteint $0,85 \pm 0,08$ de couverture - la seule stratégie au-dessus de la cible de 90 % - contre 0,72 pour la sélection gloutonne. L'avantage vient de noter les frontières par le coût de chemin Nav2 plutôt que la distance à vol d'oiseau, ce qui évite de s'engager vers une frontière proche mais séparée par un mur.

Parcours d'ingénierie. Atteindre 4/4 ne fut pas immédiat. L'exploration pure donnait une carte propre mais seulement ~ 2 victimes et révélait deux causes indépendantes cachées sous le même symptôme : la fonction de coût Nav2 décourageait le robot d'*entrer* dans les pièces

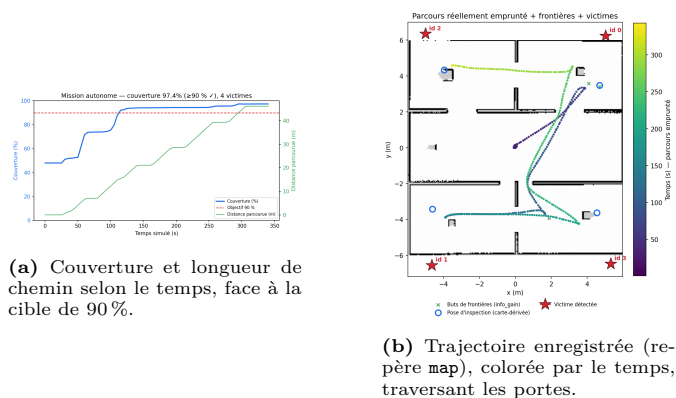


Figure 3 – Résultats quantitatifs de l'exécution finale : **97,35 %** de couverture et **4/4** victimes, entièrement autonome.

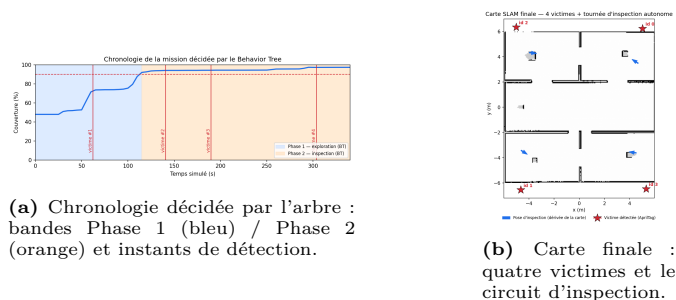


Figure 4 – Chronologie de la mission et livrable du sujet : une carte finale annotée de toutes les positions de victimes.

(un problème de couverture, corrigé en abaissant le rayon d'inflation), et un facteur temps-réel effondré *affamait le rendu de la caméra* (un problème de détection trompeur). Ajouter l'inspection en deux phases a donné 4/4 et 97 % en une exécution ; nous avons gardé un facteur temps-réel de 0,7, un réglage plus rapide coûtant une victime et sollicitant trop le GPU.

4 Conclusion

Nous avons construit une chaîne complète et conforme au sujet de recherche et sauvetage autonome autour d'un seul TurtleBot 4 : cartographie par SLAM, exploration par frontières avec critère de gain d'information, projection TF2 des détections AprilTag dans le repère global, et un arbre de comportement qui *orchestre* une mission en deux phases. La leçon clé est que la complétude de la carte et la couverture des victimes sont des objectifs distincts ; une phase d'inspection autonome dérivée de la carte les réconcilie et fait passer la détection de $\sim 2/4$ à $4/4$ tout en maintenant 97,35 % de couverture. Plus largement, le projet a renforcé trois principes : *mesurer avant de prescrire*, *une correction révèle la suivante* et *le robuste vaut mieux que le rigide*. Les perspectives incluent l'exploration RRT et un détecteur de victimes appris.

Remerciements

Nous remercions chaleureusement **Zhi Yan**, enseignant-chercheur à l'ENSTA, pour le cours IA712 et l'accompagnement qui ont rendu ce projet possible.

Références

- [1] IA712 Robotique mobile - Projet final, Projet B (sujet). <https://yzrobot.github.io/IA712/>.
- [2] B. Yamauchi, « A frontier-based approach for autonomous exploration », *IEEE CIRA*, 1997.
- [3] S. Macenski, I. Jambrecic, « slam_toolbox : SLAM for the dynamic world », *JOSS*, 2021.
- [4] S. Macenski et al., « The Marathon 2 : A navigation system (Nav2) », *IROS*, 2020.
- [5] E. Olson, « AprilTag : A robust and flexible visual fiducial system », *ICRA*, 2011.